

Choosing DNN Architectures for TinyML Tasks

Learning Goal

Given a problem description, students should be able to decide:

1. whether the task is regression, binary classification, or multiclass classification,
2. whether the input-output relationship appears linear or nonlinear,
3. what network architecture is appropriate,
4. what output activation and loss function should be used,
5. whether the design is reasonable for TinyML deployment.

Four-Step Design Rule

Step 1: Identify the task type.

Step 2: Decide whether a linear model is sufficient.

Step 3: Add hidden layers only if extra expressive power is needed.

Step 4: Match the output layer and the loss function correctly.

Activation–Loss Pairing

Task	Output Layer	Training Loss
Regression	Linear output	MSE or MAE
Binary classification	Sigmoid output <i>conceptually</i>	Binary cross entropy
Multiclass classification	Softmax output <i>conceptually</i>	Categorical cross entropy

Implementation note: In practice, it is often better to train using raw logits:

- Binary classification: use `BCEWithLogitsLoss` and do *not* place sigmoid inside the model during training.
- Multiclass classification: use `CrossEntropyLoss` and do *not* place softmax inside the model during training.

TinyML Principle

Smallest Model That Works

For TinyML, accuracy is not the only goal. We also care about:

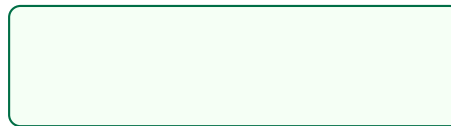
- parameter count,
- memory footprint,
- latency,
- energy consumption,
- implementation simplicity.

A model that is unnecessarily deep may work, but it is usually a poor TinyML choice.

Scenario 1: Linear Regression

Problem: 30 input features per sample, 3 continuous outputs, approximately linear relationship.

Minimal model



Recommended architecture:

$$30 \rightarrow 3$$

Output activation: Linear

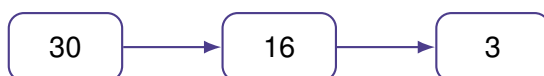
Loss: MSE or MAE

Why this is correct: If the relationship is linear, a single affine layer is enough.

Minimal vs Overparameterized Comparison



Parameters: $(30)(3) + 3 = 93$



Parameters: $(30)(16) + 16 + (16)(3) + 3 = 547$

Both models are linear maps. The deeper one uses far more parameters but does not add nonlinear expressive power.

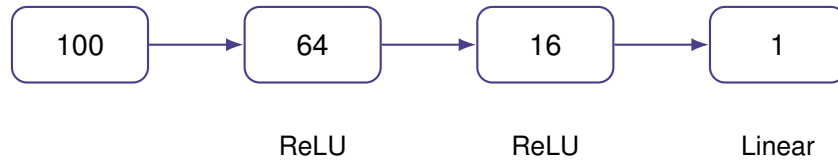
Key teaching point: A stack of linear layers is still just a linear transformation. So here,

$$30 \rightarrow 16 \rightarrow 3$$

with linear activations is unnecessarily expensive.

Scenario 2: Nonlinear Regression

Problem: 100 input features, 1 continuous output, nonlinear relationship.



Recommended architecture:

$$100 \rightarrow 64 \rightarrow 16 \rightarrow 1$$

Output activation: Linear

Loss: MSE or MAE

Why this is correct: Since the mapping is nonlinear, hidden layers with nonlinear activations are useful.

Parameter count:

$$(100)(64) + 64 + (64)(16) + 16 + (16)(1) + 1 = 7521$$

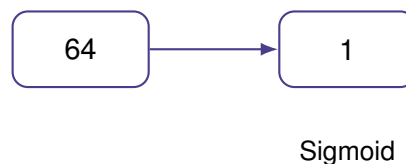
TinyML note: This may still be too large for a tight microcontroller budget. A smaller alternative such as

$$100 \rightarrow 32 \rightarrow 1$$

may be worth testing first.

Scenario 3: Binary Classification, Linearly Separable

Problem: 64 input features, 1 binary output, approximately linearly separable.



Recommended architecture:

$$64 \rightarrow 1$$

Output activation: Sigmoid *conceptually*

Loss: Binary cross entropy

Practical training note: Prefer raw logits plus BCEWithLogitsLoss during training.

Why this is correct: If the data is linearly separable, logistic regression is usually sufficient.

Minimal vs Overparameterized Comparison



Parameters: $(64)(1) + 1 = 65$

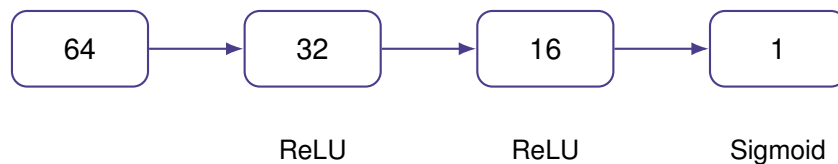


Parameters: $(64)(16) + 16 + (16)(1) + 1 = 1057$

If the hidden layer is linear, then the model still behaves like logistic regression after collapsing the linear transformations before the final sigmoid.

Scenario 4: Binary Classification, Nonlinear Boundary

Problem: 64 input features, 1 binary output, not linearly separable.



Recommended architecture:

$$64 \rightarrow 32 \rightarrow 16 \rightarrow 1$$

Output activation: Sigmoid *conceptually*

Loss: Binary cross entropy

Practical training note: Prefer raw logits plus BCEWithLogitsLoss during training.

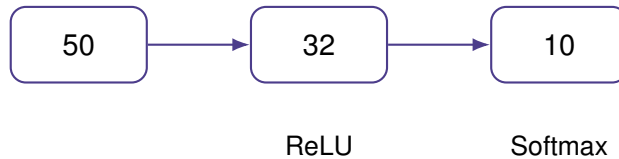
Parameter count:

$$(64)(32) + 32 + (32)(16) + 16 + (16)(1) + 1 = 2625$$

Why this is correct: Now the hidden nonlinearities let the model learn a nonlinear decision boundary.

Scenario 5: Multiclass Classification

Problem: 50 input features, 10 output classes, nonlinear class boundaries.



Recommended architecture:

$$50 \rightarrow 32 \rightarrow 10$$

Output activation: Softmax *conceptually*

Loss: Categorical cross entropy

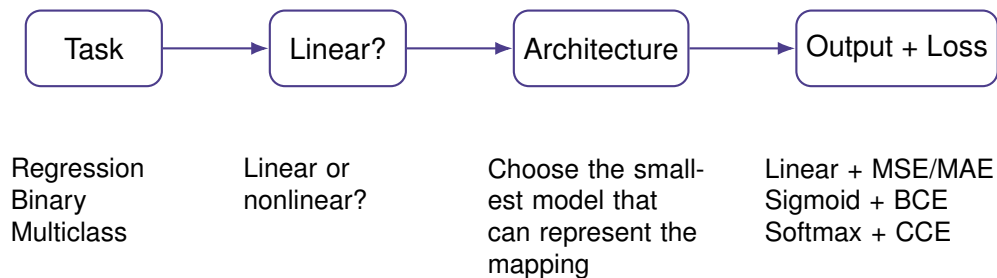
Practical training note: Prefer raw logits plus `CrossEntropyLoss` during training.

Parameter count:

$$(50)(32) + 32 + (32)(10) + 10 = 1962$$

Why this is correct: This is a standard feedforward setup for multiclass classification with moderate nonlinear complexity.

Summary Figure



Final Message for Students

Do not ask, “How deep can I make the network?”

Ask instead, “What is the smallest architecture that correctly matches the task and is still deployable on-device?”